Corei3 Radiation Mitigation Supervisor

Technical Architecture Document

SPEC 26 Space Tech Hackathon
Team Corei3
NED University of Engineering and Technology
26 October 2026
Version 1.0

Purpose and implementation status

Corei3 combines triple modular redundancy, an independently clocked watchdog, and SPI readable telemetry to demonstrate fault masking, safe fallback, and controlled reset recovery. This document defines the implemented interfaces and explains the evidence and assumptions needed to assess the Level 3 submission.

The project contains a C software model, synthesizable Verilog modules, self checking HDL testbenches, and a local browser dashboard. Functional simulation is verified. FPGA synthesis, timing closure, board operation, and radiation qualification have not been demonstrated.

Document navigation

Page	Content
2	System architecture and module responsibilities
3	Voting and safety control
4	Top level pins and configuration
5	SPI protocol and timing
6	Read only register map
7	Status flags and event semantics
8	Dashboard and simulation integration
9	Verification and fault scenarios
10 to 13	Simulation results and evidence with screenshot spaces
14	Operation and implementation limits

1 System architecture

The protected computation produces three coherent 8 bit words. A bitwise voter produces the raw result. A safety multiplexer either forwards that result or selects the inactive command zero. Watchdog timeout and integrity alarms latch the multiplexer into safe mode.

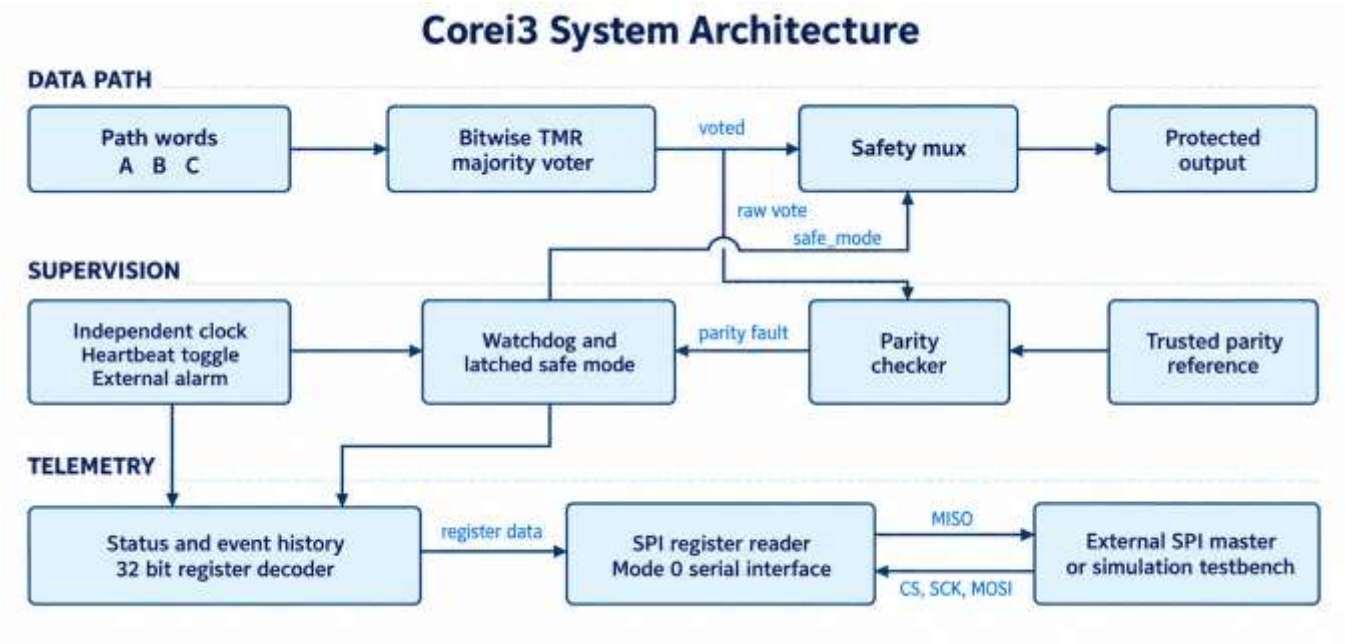


FIGURE 1 IMPLEMENTED LEVEL 3 DATA PATH SUPERVISION AND SPI TELEMETRY

Module responsibilities

Source module	Implemented responsibility
tmr_voter.v	Bitwise majority vote and path disagreement indication.
watchdog.v	Heartbeat synchronization, elapsed timer, and sticky timeout and integrity latches.
supervisor.v	Voter and watchdog integration; protected output selects voted or SAFE_VALUE.
telemetry_supervisor.v	8 bit Level 3 wrapper; parity checker, event counters, history, register decoder and SPI instance.
spi_register_reader.v	Oversampled Mode 0 slave for one 8 bit address and one 32 bit response per frame.

Zero is an example inactive actuator command. A deployed system must choose its safe behavior from the controlled device and mission requirements. Raw voted remains available during safe mode for diagnosis.

2 Voting and safety control

Direct input model and fault injection

For the demonstration, each redundant path receives the same unsigned value x from 0 through 255. The C model uses f(x) = x. The simulation applies independent XOR masks to the three path words. These are logical redundant paths, not three physical computers.

```
A = x XOR mask_A
B = x XOR mask_B
C = x XOR mask_C
voted = (A & B) | (A & C) | (B & C)
```

With x = 10, the healthy word is 00001010. Flipping bit 0 in A produces 00001011, or 11. B and C preserve the voted output 10. Voting masks the fault at the output; it does not repair the faulty word. Multiple corrupted bits in one path are also masked if the other two paths remain correct.

Independent watchdog

The processor toggles heartbeat_toggle_async after useful completed work. Two flip flops synchronize the toggle into wd_clk; an additional delayed sample detects a change. A recognized toggle clears elapsed. If no recognized toggle arrives for TIMEOUT_CYCLES ticks, timeout_latched asserts and elapsed saturates at the timeout value. A stuck high or stuck low heartbeat is not continuing progress.

The Level 3 wrapper and combined dashboard use 64 ticks. The standalone watchdog module defaults to 16 ticks, and the separate Level 2 regression explicitly tests that setting. At a 10 ns simulation clock period, 64 ticks correspond to 640 ns from the last accepted heartbeat or reset release. Asynchronous heartbeat recognition adds synchronization latency.

Parity and external integrity alarm

```
parity_fault = (^voted) != expected_parity
core_integrity_alarm = integrity_fault | parity_fault
safe_mode = timeout_latched | integrity_latched
```

The expected parity must come from a trusted independent source. The testbench derives it from the known healthy input, representing this external reference. An odd number of wrong output bits changes parity; an even number can preserve it. An external integrity monitor may assert integrity_fault, but no memory scanner, CRC engine, or ECC engine is implemented here.

Fallback and controlled recovery

An asserted integrity alarm latches at a watchdog clock edge. Either latch selects safe output zero. Restarting heartbeat or removing an alarm does not clear the latch. reset_n clears both safety latches and telemetry. The raw HDL reset does not repair or replace external path data; the dashboard reset action separately removes injected masks and resumes heartbeat.

3 Top level pins and configuration

The Level 3 top level is telemetry_supervisor. All data words are unsigned. The table describes logical ports; no physical FPGA package pins are assigned.

Port	Dir	Bits	Definition
wd_clk	In	1	Independent free running watchdog and telemetry clock.
reset_n	In	1	Active low asynchronous reset for latches, counters and SPI state.
heartbeat_toggle_async	In	1	Processor progress toggle; synchronized inside the watchdog.
integrity_fault	In	1	External corruption alarm synchronous to wd_clk.
a, b, c	In	8 each	Three coherent redundant path values.
expected_parity	In	1	Trusted XOR parity of the intended valid output.
spi_cs_n	In	1	Active low SPI chip select.
spi_sck	In	1	Mode 0 clock with idle level zero.
spi_mosi	In	1	Master data; address transmitted most significant bit first.
spi_miso	Out	1	Serial response; high impedance while chip select is inactive.
voted	Out	8	Raw bitwise majority result.
protected_output	Out	8	Raw voted value, or zero while safe mode is latched.
disagreement	Out	1	At least two path words differ.
safe_mode	Out	1	Logical OR of the sticky timeout and integrity latches.
parity_fault	Out	1	Raw voted parity differs from expected_parity.
no_word_majority	Out	1	All three complete path words differ.

Configuration and integration contract

TIMEOUT_CYCLES defaults to 64 in telemetry_supervisor; supported configuration is an integer from 1 through 2^31 - 1. The reusable supervisor provides WIDTH and SAFE_VALUE parameters, but this telemetry wrapper fixes the data width to 8 and safe value to zero.

Provide a genuinely independent watchdog clock on hardware. Synchronize reset release in the wd_clk domain. Path words and expected parity must be coherent and stable at sampling; the combinational voter is not a multi bit clock domain synchronizer. A real processor interface needs a stable sampled transfer or handshake.

4 SPI protocol and timing

The interface is a read only custom register protocol using SPI Mode 0 timing. CPOL is zero, CPHA is zero, and both address and data are transmitted most significant bit first. SPI inputs are synchronized and oversampled in wd_clk; SCK is not used as a generated HDL clock.

Frame phase	Master action	Slave response
Select	Assert CS low while SCK is low.	Prepare a new frame.
Address	Transmit one 8 bit byte address on MOSI.	Capture the addressed 32 bit register at the final address bit.
Data	Transmit 32 dummy zero bits. Sample MISO on rising SCK edges.	Return response bits 31 through 0; subsequent data bits advance after falling SCK edges.
End	Deassert CS after the response.	Release MISO and reinitialize frame state once deselection is observed.

Each selected frame contains one address and one response, for 40 SCK cycles. No command prefix, writes, bursts, auto increment, or remote reset command is supported. Unmapped or misaligned addresses return 0xDEADBEEF. Aborted frames are discarded after chip select deselection is synchronized.

Required timing margin

Timing condition	Minimum requirement
SCK high and low intervals	Each at least 8 wd_clk periods.
CS setup before first rising SCK	At least 8 wd_clk periods.
CS inactive between frames	At least 8 wd_clk periods.
MOSI setup	Stable at least 4 wd_clk periods before rising SCK; hold through that edge.
Maximum SCK frequency	wd_clk frequency divided by 16, or slower.

These margins account for oversampling latency. MISO is valid after the corresponding synchronized edge is processed; the master samples the response on the following data rising edge. The testbench uses slower compliant timing. In a target FPGA, implement MISO high impedance with an appropriate top level I/O buffer or separate data and output enable signals.

Snapshot semantics and example

A read captures one complete 32 bit register, so its response does not tear if live signals change during the 32 data clocks. Separate register reads are not a global atomic snapshot. The dashboard records the action number associated with each read.

```
MOSI:  00000100  followed by 32 dummy zeros
Read:  STATUS at byte address 0x04
MISO response during timeout only: 0x00000003
```

5 Read only register map

All register responses are 32 bits. Addresses below are byte addresses with 4 byte spacing. Reserved bits return zero. The timeout register reports the configured parameter; it cannot be changed over SPI.

Address	Register	Reset value	Definition
0x00	ID	0x43334933	Fixed Corei3 implementation identifier.
0x04	STATUS	Input dependent	Current flags and sticky safety causes. See page 7.
0x08	MISMATCH_EVENTS	0	Rising path disagreement episodes.
0x0C	PARITY_EVENTS	0	Rising parity fault episodes.
0x10	AMBIGUOUS_EVENTS	0	Rising no_word_majority episodes.
0x14	PATH_SNAPSHOT	External data	[31:24] C; [23:16] B; [15:8] A; [7:0] voted.
0x18	PROTECTED_OUTPUT	Voted	Protected result in [7:0]; [31:8] zero. Reset clears the safety latch.
0x1C	ELAPSED	0	Watchdog elapsed ticks; saturates at timeout.
0x20	HISTORY	0	Sticky historical flags. See page 7.
0x24	TIMEOUT	64 default	Configured TIMEOUT_CYCLES value.
0x28	VERSION	0x00010000	Register interface version 1.0.

Reset clears stored state. STATUS still reflects combinational flags and external alarm inputs. PATH_SNAPSHOT reflects the external path inputs rather than an internally reset data store. With default safe value zero, PROTECTED_OUTPUT follows voted when reset clears safe mode; the system reset controller must hold actuators inactive if that is required during initialization.

Example reads for input 10

Condition	Register	Expected response
Healthy A = B = C = 10	PATH_SNAPSHOT	0x0A0A0A0A
Healthy operation	PROTECTED_OUTPUT	0x0000000A
Timeout only	STATUS	0x00000003
A = B = 11 and C = 10 with valid parity reference	STATUS	0x0000002D after integrity latch sets
Unmapped address 0xFF	Any undefined register	0xDEADBEEF

6 Status flags and event semantics

STATUS register at 0x04

Bit	Signal	Meaning when set
0	safe_mode	Protected output selects zero.
1	timeout_latched	Missing heartbeat triggered the watchdog.
2	integrity_latched	Parity fault or external integrity alarm has latched.
3	disagreement	Not all path words agree.
4	no_word_majority	A, B and C are all different complete words.
5	parity_fault	Current raw voted parity differs from the trusted reference.
6	integrity_fault	External monitor alarm is currently asserted.
7 to 31	Reserved	Always zero.

HISTORY register at 0x20

Bit	Historical event retained until reset
0	Watchdog timeout latch observed.
1	Integrity latch observed.
2	Path disagreement observed.
3	No complete word majority observed.
4	Parity fault observed.
5	External integrity alarm observed.
6 to 31	Reserved zero.

Latch derived history may lag current status by one wd_clk tick because sequential blocks sample the previous register values. Reading HISTORY does not clear it. Current combinational status and latched causes remain separately visible.

Counters and interpretation

The three 32 bit event counters increment on a sampled low to high transition of their corresponding condition. A condition that stays high counts once. A sampled deassertion followed by a new assertion counts again. Counters saturate at 0xFFFFFFFF and clear on reset; they do not wrap.

Disagreement does not prove that the majority is right. Likewise, no_word_majority is an uncertainty flag: all three whole words can differ while bitwise voting remains correct. Neither condition alone triggers safe mode. Hardware counters record fault episodes, not proven corrections. Level 1 correction counts require the software test oracle.

7 Dashboard and simulation integration

The combined browser interface controls the C model and HDL simulation through a local Node.js server. HTML, CSS and JavaScript provide presentation and controls; the voting and supervisor results come from executed C and Verilog, not browser calculations.

Execution sequence

1. At startup, server.js compiles tmr.c with GCC and the three HDL testbench targets with Icarus Verilog. A compilation failure stops startup.

2. Each browser session stores an ordered sequence of validated actions. Actions change input and XOR masks, advance watchdog ticks, change heartbeat or integrity alarm state, reset the model, or clock an SPI read.

3. tb_dashboard.v replays the session from reset in a new simulator process. Clocked RTL reconstructs counters and sticky safety deterministically. The combined bridge instantiates Level 2 and Level 3 supervisors side by side.

4. After an action, the server executes the C model and compares the voted value and path values with the HDL response. The frontend displays cross check status and records simulated time.

5. SPI requests drive actual CS, SCK and MOSI signals and capture MISO. The exported evidence contains session actions, returned states, register snapshots and any test suite output.

Time and reset behavior

Simulation advances when an action executes, not while the browser is idle. SPI transactions also consume watchdog ticks, so reading telemetry while heartbeat is stopped can itself advance the model into timeout. Healthy heartbeat continues toggling during SPI reads when enabled. The default bridge heartbeat toggles every five watchdog ticks.

Clear faults removes data masks without clearing safety latches. Controlled reset performs an HDL reset and separately restores healthy testbench inputs, clears injected alarms, resumes heartbeat, and clears displayed SPI snapshots. Session evidence and software correction totals remain. New session or page reload starts fresh session evidence.

Dashboard counters and evidence

Oracle verified corrected events count only fault setting transactions with disagreement and a vote equal to the intended input. Corrected bit instances sum the erroneous path bits masked in those successful transactions. SPI reads and time advance actions do not inflate these totals. Hardware episode counters follow the different semantics defined on page 7.

Each session permits up to 500 actions. The log displays the newest 100; JSON export retains all recorded session actions. Stored register rows are individually timestamped snapshots, not live simultaneous readings. The architecture link exposes the accompanying register definitions.

8 Verification and fault scenarios

The supplied execution evidence records zero failures in the model, RTL and integration checks. Tests validate the behaviors listed below; the counts are not a certification of every possible hardware failure.

Suite	Checks	Evidence
C Level 1 model	7,180	Truth table; 6,144 single bit injections; one path mask sweep; limits and fault removal.
Verilog Level 2	6,932	Voting; heartbeat operation; hangs; stuck heartbeat; timeout boundary; integrity input; sticky safety and reset.
Verilog and SPI Level 3	31	Bit serial register reads; counters; parity coverage and blind spot; CS abort recovery; atomic register response.
Combined backend integration	50	HTTP controls through C and RTL, SPI during hang, validation, independent sessions and all three verification suites.

The three model and HDL suites total 14,143 checks; integration adds 50. Testbench stimulus includes faults outside the guaranteed correction envelope so that successful tests also demonstrate known failures of TMR or parity.

Representative scenarios for input 10

Injected condition	Observed behavior
A = 11; B = C = 10	Vote 10; disagreement asserted; both supervisors remain normal.
A = 5; B = C = 10	Four wrong bits in A are masked; vote remains 10.
A = 11; B = 8; C = 10	All words differ, yet bitwise vote is 10; ambiguity flag is reported.
A = B = 11; C = 10	Vote 11 is wrong. Level 2 alone forwards it; Level 3 parity latches safe output zero.
A = B = C = 11	No path disagreement, but independent parity detects the odd bit output error.
A = B = 9; C = 10	Wrong vote 9 preserves parity. Hardware does not detect it; the test oracle exposes the blind spot.
Heartbeat stops	Both supervisors time out and force zero; SPI remains responsive.
Fault removed or heartbeat resumes	Safety remains latched until controlled reset.

Browser verification exercised the nine step guided demo, bit toggles, correction counters, all eleven SPI reads, verification result cards and evidence export. Evidence files are included in the combined project archive.

9 Simulation results and evidence

Single bit fault masking

Set input to 10, click Controlled reset, then Single bit A. Capture all three paths and the majority decision panel.

Required values: A = 11, B = 10, C = 10; voted output = 10; status FAULT MASKED.

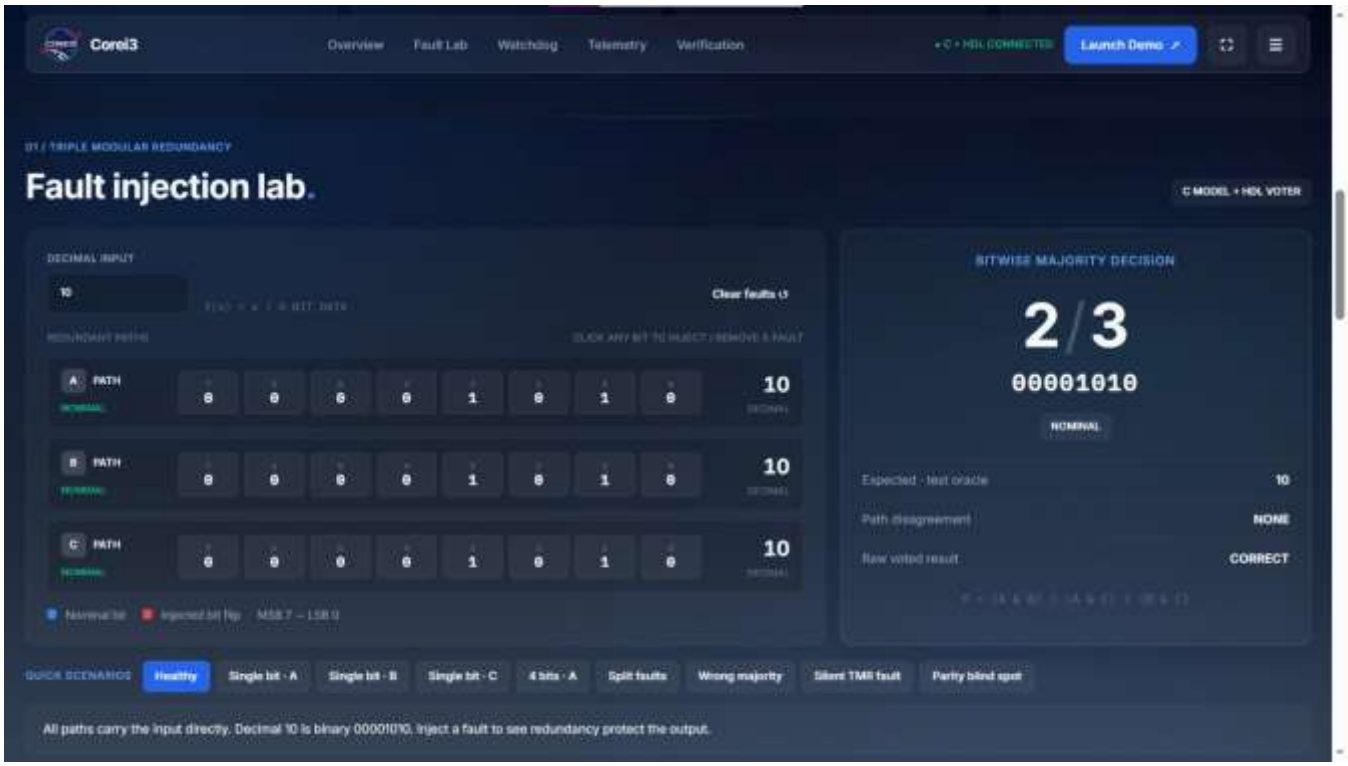


FIGURE 2 SINGLE BIT FAULT MASKING

Path A changes from 10 to 11, while paths B and C remain 10. The majority voter preserves the correct output of 10.

Watchdog timeout and safe mode

Click Controlled reset, Stop heartbeat, then Advance 64 ticks. Capture the Watchdog timeline and Safety supervisor panels together.

Required values: elapsed = 64; TIMEOUT LATCHED; both supervisors show SAFE / OUTPUT 0.

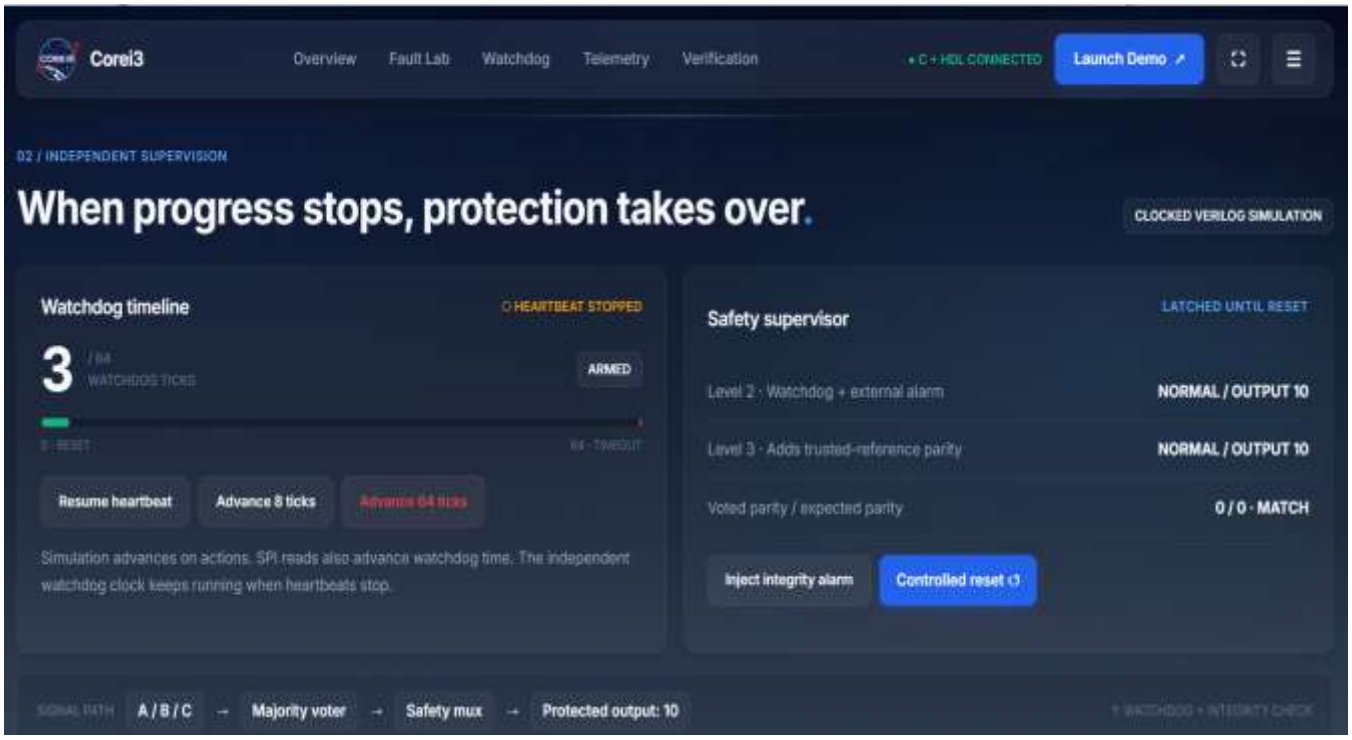


FIGURE 3 WATCHDOG TIMEOUT AND SAFE FALLBACK

When processor heartbeats stop, the watchdog reaches its 64 tick limit. Both supervisors latch safe mode and force their protected outputs to zero.

9 Simulation results and evidence continued

SPI telemetry during a processor hang

Keep the timeout state from Figure 3. Read STATUS at address 0x04. Capture the STATUS row and Last SPI Transaction panel together.

Required values: address 0x04; response 0x00000003; decoded flags SAFE MODE and TIMEOUT.

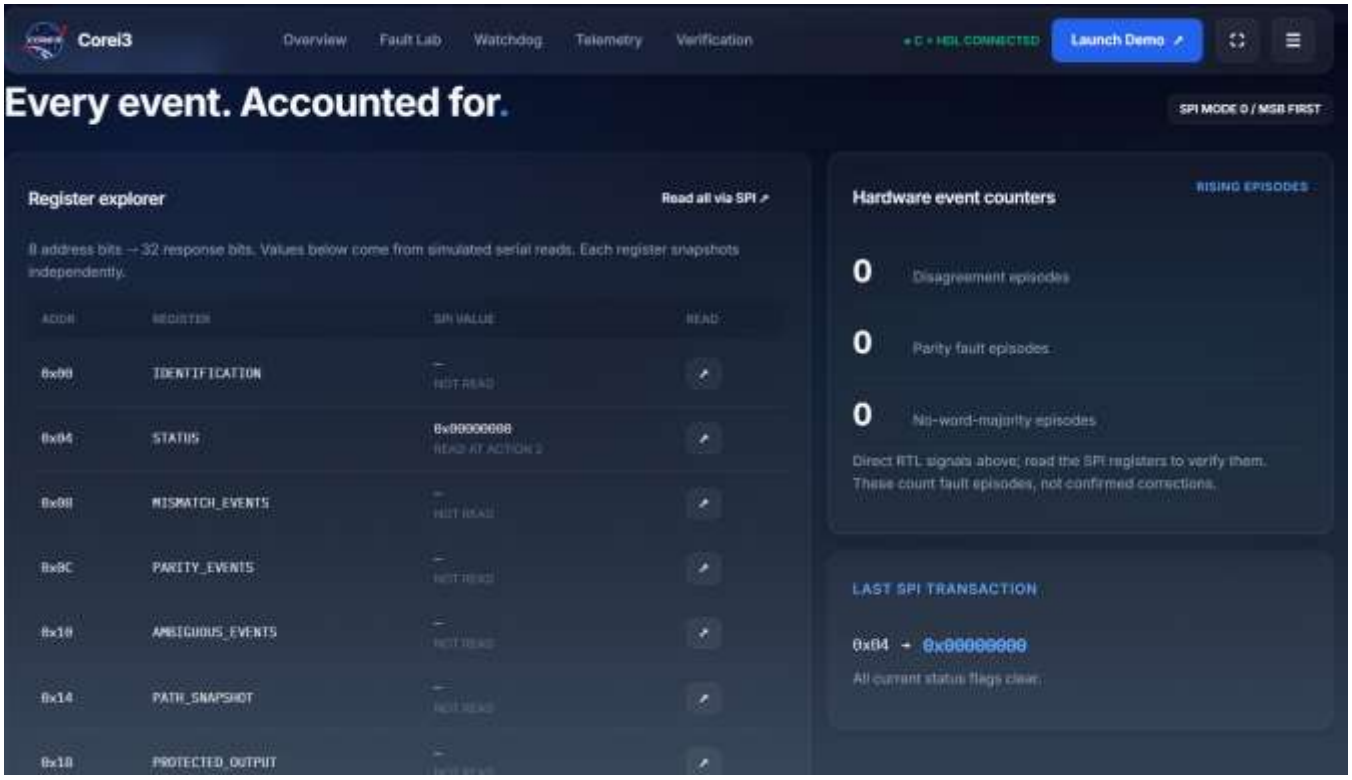


FIGURE 4 SPI STATUS READ DURING A PROCESSOR HANG

The SPI STATUS response is 0x00000003, confirming safe mode and watchdog timeout. Telemetry remains accessible while processor heartbeats are stopped.

9 Simulation results and evidence continued

Automated verification results

Click Run all verification suites and wait for completion. Capture the three verification cards; include the console output if it is readable.

Required results: C model 7,180 checks; Level 2 HDL 6,932 checks; Level 3 SPI HDL 31 checks; all PASS.

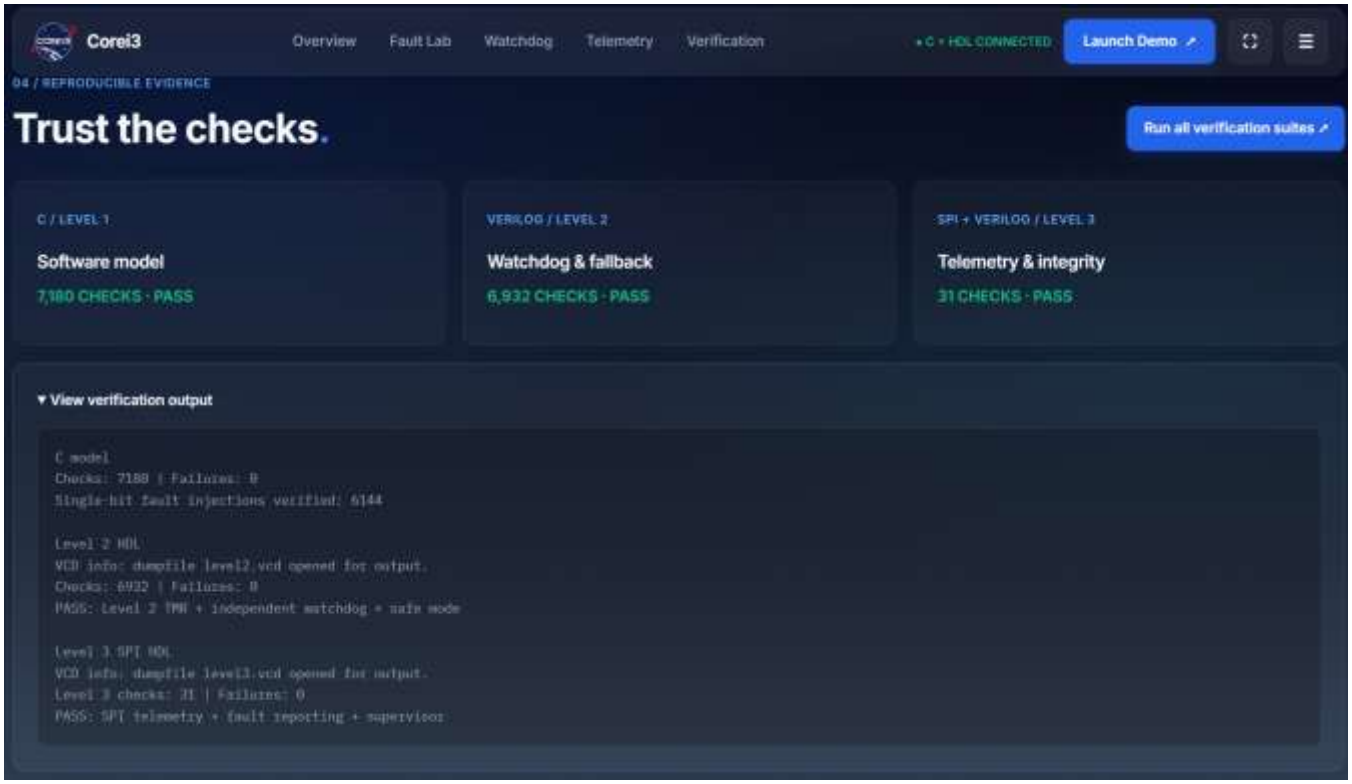


FIGURE 5 AUTOMATED VERIFICATION RESULTS

The C model, Level 2 HDL, and Level 3 SPI suites complete 14,143 checks with zero failures. The separate backend integration suite contains another 50 checks.

10 Operation and implementation limits

Reproduce the local demonstration

Prerequisites are Node.js, GCC, and Icarus Verilog with iverilog and vvp available. GTKWave is optional for waveform inspection. The launcher recognizes the existing Windows Icarus installation at D:\iverilog\bin. No npm packages or internet connection are required at runtime.

```
1. Stop any older dashboard using port 8080.
2. Extract Corei3_All_Levels.zip into a new folder.
3. Run start.bat or node server.js.
4. Open http://127.0.0.1:8080 in a browser.
5. Launch the guided demo, run verification, and export evidence.
```

The server binds to the local loopback interface by default. To run the backend integration suite, stop the dashboard first and execute node test-integration.js in the project folder. The tests start their own server. Standalone HDL testbench sources are in sim; RTL sources are in rtl.

Assumptions and remaining implementation work

The demonstration assumes a healthy voter, watchdog clock, reset mechanism, coherent path inputs, and a trustworthy parity reference. Faults are injected into path output values; this is not a model of every FPGA configuration upset. Transient alarms must persist long enough to be sampled by wd_clk.

TMR can fail when more than one copy is wrong at the same bit. Parity can miss an even number of wrong output bits or a corrupted reference. A watchdog checks progress, not arbitrary correctness. No general memory corruption detector, ECC scrubber, trusted program reload, autonomous restart, or physical processor core is implemented.

Before a board deployment, select the actual safe actuator behavior, provide the independent clock, implement reset release synchronization and data coherency, protect the integrity reference, add timing and pin constraints, synthesize and review the design, and validate the SPI I/O buffer and timing on the chosen FPGA. Radiation tolerance requires separate qualification.

Implementation records

Source files: tmr.c ; rtl/tmr_voter.v ; rtl/watchdog.v ; rtl/supervisor.v ; rtl/telemetry_supervisor.v ; rtl/spi_register_reader.v ; sim/tb_dashboard.v ; sim/tb_supervisor.v ; sim/tb_level3.v ; server.js ; test-integration.js ; evidence/integration.txt.

Browser evidence: evidence/browser-checks.txt and the sample export evidence/browser-session.json.

Submission contents: Code, fault-injection testbenches, pin and register definitions, and the guided demonstration script.

Summary: The design demonstrates functional Level 3 behavior in simulation while stating its detection and recovery limits explicitly.